

Django システム開発演習

■ Django アプリケーション作成の環境構築

[ワークスペースの作成] : 作成しなくてもいい

任意の場所（デスクトップや SSD 等）に「django_workspace」フォルダを作成

→共有 PC を使用しているのでワークスペースは各々やりやすいように任せます

構成

```
django_workspace
|
|
```

[プロジェクトを作成] : プロジェクト名は【django_project】

実行 CMD : django-admin startproject プロジェクト名」

```
> django-admin startproject django_project
```

構成

```
django_workspace
|
|-django_project
|
|   |-django_project
|   |
|   |-manage.py
|-venv
```

[アプリケーションの作成]

1.カレントディレクトリを「django_project」に移動（CD 等で移動）

実行 CMD > CD django_project

2.アプリケーションの作成コマンドを実行

書式「python manage.py startapp アプリケーション名」

実行 CMD > python manage.py startapp django_book_store

構成

```
django_workspace
|
|--django_project
|
|   |--django_project
|   |
|   |--manage.py
|   |
|   |-- django_book_store
|   |
|   |   |--migrations
|   |   |
|   |   |--__init__.py
|   |   |
|   |   |--admin.py
|   |   |
|   |   |--apps.py
|   |   |
|   |   |--models.py
|   |   |
|   |   |--tests.py
|   |   |
|   |   |--views.py
|--venv
```

[サーバーの起動と確認]

1. ターミナルでサーバーの起動コマンドを実行

```
> python manage.py runserver
```

2. ブラウザを起動し下記にアクセスしてサーバーの起動を確認

```
url : localhost:8000
```

「インストールは正常に機能しました！おめでとう！」と表示されるのを確認。

[テーブル情報]

Employee 表の完成イメージ

employee_no	employee_name	password
R20001	鈴木一郎	ry000001
R20002	山田太郎	rx000002
R20003	坂本竜馬	rw000003
R20004	田中花	rv000004

4. App の登録

settings.py の編集

5. Model の作成

6. マイグレーション

```
> python manage.py makemigrations django_book_store
Migrations for 'psys':
  psys¥migrations¥0001_initial.py
    - Create model AuthGroup
    - Create model AuthGroupPermissions
    - Create model AuthPermission
    - Create model AuthUser
    - Create model AuthUserGroups
```

- Create model AuthUserUserPermissions
- Create model Customer
- Create model CustomerNumbering
- Create model DjangoAdminLog
- Create model DjangoContentType
- Create model DjangoMigrations
- Create model DjangoSession
- Create model Employee
- Create model Item
- Create model Orders
- Create model OrderDetails

マイグレーションファイルを確認します

```
from django.db import migrations, models
import django.db.models.deletion
```

```
class Migration(migrations.Migration):
    initial = True
    dependencies = [
        ('psys', '0001_initial'),
    ]
    operations = [
        migrations.CreateModel(
            name='Customer',
            fields=[
                ('customer_code', models.CharField(max_length=6,
primary_key=True, serialize=False)),
                ('customer_name', models.CharField(blank=True, max_length=32,
null=True)),
                ('customer_telno', models.CharField(blank=True, max_length=13,
null=True)),
```

```
        ('customer_postalcode', models.CharField(blank=True,
max_length=8, null=True)),
        ('customer_address', models.CharField(blank=True,
max_length=40, null=True)),
        ('discount_rate', models.IntegerField(blank=True, null=True)),
        ('delete_flag', models.IntegerField()),
    ],
    options={
        'db_table': 'customer', # データベース上の実際のテーブル名

        'managed': False, # Django がこのテーブルの変更を管理しない
    },
```

以降省略

マイグレートします

```
>python manage.py migrate
```

Operations to perform:

Apply all migrations: admin, auth, contenttypes, psys, sessions

Running migrations:

Applying psys.0001_initial... OK

7. プロトタイプ開発とその後の流れ

お客様の要求は下記

「まずは得意先検索機能をプロトタイプとして開発し、

各機能の必要性と各機能を検討して、1次開発として開発すべき機能を提案する」

(1) 設計書 (ver.1) を基に「得意先検索機能」をプロトタイプとして開発 (→確認)

(2) 設計書 (ver.1) を基に「得意先管理システム」をプロトタイプ開発 (→確認)

(3) 1次開発として開発すべき機能を提案書 (様式は自由) にまとめる (→確認)

(4) 提案書を基に設計書 (ver.2) を作成 (→確認)

(5) 設計書 (ver.2) を基に「得意先管理システム」を完成 (→確認)

(6) 要求仕様書を基に「得意先別集計システム」の設計書 (ver.3) を作成 (→確認)

(7) 設計書 (ver.3) 基に「得意先別集計システム」を完成 (→確認)

(8) 従業員の新規登録機能を追加 (設計書 ver.4) (→確認)

※誰が登録できるようにするのか？

(9) その他追加した方が良い機能などあれば改良 (設計書 ver.5) (→確認)

(10) 設計書を仕上げよう (誰が見ても同じようなものができるように) (→確認)